

There is no such thing as scope for attackers. Therefore, ASVS requirements should be evaluated alongside guidance for other aspects of the application lifecycle, including CI/CD processes, hosting, and operational activities.

Application

ASVS defines an “application” as the software product being developed, into which security controls must be integrated. ASVS does not prescribe development lifecycle activities or dictate how the application should be built via a CI/CD pipeline; instead, it specifies the security outcomes that must be achieved within the product itself.

Components that serve, modify, or validate HTTP traffic, such as Web Application Firewalls (WAFs), load balancers, or proxies, may be considered part of the application for those specific purposes, as some security controls depend directly on them or can be implemented through them. These components should be considered for requirements related to cached responses, rate limiting, or restricting incoming and outgoing connections based on source and destination.

Conversely, ASVS generally excludes requirements that are not directly relevant to the application or where configuration is outside the application’s responsibility. For example, DNS issues are typically managed by a separate team or function.

Similarly, while the application is responsible for how it consumes input and produces output, if an external process interacts with the application or its data, it is considered out of scope for ASVS. For instance, backing up the application or its data is usually the responsibility of an external process and is not controlled by the application or its developers.

Security

Every requirement must have a demonstrable impact on security. The absence of a requirement must result in a less secure application, and implementing the requirement must reduce either the likelihood or the impact of a security risk.

All other considerations, such as functional aspects, code style, or policy requirements, are out of scope.

Verification

The requirement must be verifiable, and the verification must result in a “fail” or “pass” decision.

Standard

The ASVS is designed to be a collection of security requirements to be implemented to comply with the standard. This means that requirements are limited to defining the security goal to achieve that. Other related information can be built on top of ASVS or linked via mappings.

Specifically, OWASP has many projects, and the ASVS deliberately avoids overlapping with the content in other projects. For example, developers may have a question, “how do I implement a particular requirement in my particular technology or environment,” and this should be covered by the Cheat Sheet Series project. Verifiers may have a question “how do I test this requirement in this environment,” and this should be covered by the Web Security Testing Guide project.

Whilst the ASVS is not just intended for security experts to use, it does expect the reader to have technical knowledge to understand the content or the ability to research particular concepts.

Requirement

The word requirement is used specifically in the ASVS as it describes what must be achieved to satisfy it. The ASVS only contains requirements (must) and does not contain recommendations (should) as the main condition.

In other words, recommendations, whether they are just one of many possible options to solve a problem or code style considerations, do not satisfy the definition to be a requirement.

ASVS requirements are intended to address specific security principles without being too implementation or technology-specific, at the same time, being self-explanatory as to why they exist. This also means that requirements are not built around a particular verification method or implementation.

Documented security decisions

In software security, planning security design and the mechanisms to be used early on will lead to a more consistent and reliable implementation in the finished product or feature.

Additionally, for certain requirements, implementation will be complicated and very specific to an application’s needs. Common examples include permissions, input validation, and protective controls around different levels of sensitive data.

To account for this, rather than sweeping statements like “all data must be encrypted” or trying to cover every possible use case in a requirement, documentation requirements were included which mandate that the application developer’s approach and configuration to these sorts of controls must be documented. This can then be reviewed for appropriateness and then the actual implementation can be compared to the documentation to assess whether the implementation matches expectations.

These requirements are intended to document the decisions which the organization developing the application has taken regarding how to implement certain security requirements.

Documentation requirements are always in the first section of a chapter (although not every chapter has them) and always have a related implementation requirement where the decisions that are documented should actually be put into place. The point here is that verifying that the documentation is in place and that the actual implementation are two separate activities.

There are two key drivers for including these requirements. The first driver is that a security requirement will often involve enforcing rules e.g., what kind of file types are allowed to be uploaded, what business controls should be enforced, what are the allowed characters for a particular field. These rules will differ for every application, and therefore, the ASVS cannot prescriptively define what they should be, nor will a cheat sheet or more detailed response help in this case. Similarly, without these decisions being documented, it will not be possible to perform verification of the requirements that implement these decisions.

The second driver is that for certain requirements, it is important to provide an application development with flexibility regarding how to address particular security challenges. For example, in previous ASVS versions, session timeout rules were very prescriptive. Practically speaking, many applications, especially those that are consumer-facing, have much more relaxed rules and prefer to implement other mitigation controls instead. Documentation requirements, therefore, explicitly allow for flexibility around this.

Clearly, it is not expected that individual developers will be making and documenting these decisions but rather the organization as a whole will be taking those decisions and making sure that they are communicated to developers who then make sure to follow them.

Providing developers with specifications and designs for new features and functionality is a standard part of software development. Similarly, developers are expected to use common components and user interface mechanisms rather than just making their own decisions each time. As such, extending this to security should not be seen as surprising or controversial.

There is also flexibility around how to achieve this. Security decisions might be documented in a literal document, which developers are expected to refer to. Alternatively, security decisions could be documented and implemented in a common code library that all developers are mandated to use. In both cases, the desired result is achieved.

Application Security Verification Levels

The ASVS defines three security verification levels, with each level increasing in depth and complexity. The general aim is for organizations to start with the first level to address the most critical security concerns, and then move up to the higher levels according to the organization and application needs. Levels may be presented as L1, L2, and L3 in the document and in requirement texts.

Each ASVS level indicates the security requirements that are required to achieve from that level, with the higher remaining level requirements as recommendations.

In order to avoid duplicate requirements or requirements that are no longer relevant at higher levels, some requirements apply to a particular level but have more stringent conditions for higher levels.

Level evaluation

Levels are defined by priority-based evaluation of each requirement based on experience implementing and testing security requirements. The main focus is on comparing risk reduction with the effort to implement the requirement. Another key factor is to keep a low barrier to entry.

Risk reduction considers the extent to which the requirement reduces the level of security risk within the application, taking into account the classic Confidentiality, Integrity, and Availability impact factors as well as considering whether this is a primary layer of defense or whether it would be considered defense in depth.

The rigorous discussions around both the criteria and the leveling decisions have resulted in an allocation which should hold true for the vast majority of cases, whilst accepting that it may not be a 100% fit for every situation. This means that in certain cases, organizations may wish to prioritize requirements from a higher level earlier on based on their own specific risk considerations.

The types of requirements in each level could be characterized as follows.

Level 1

This level contains the minimum requirements to consider when securing an application and represents a critical starting point. This level contains around 20% of the ASVS requirements. The goal for this level is to have as few requirements as possible, to decrease the barrier to entry.

These requirements are generally critical or basic, first-layer of defense requirements for preventing common attacks that do not require other vulnerabilities or preconditions to be exploitable.

In addition to the first layer of defense requirements, some requirements have less of an impact at higher levels, such as requirements related to passwords. Those are more important for Level 1, as from higher levels, the multi-factor authentication requirements become relevant.

Level 1 is not necessarily penetration testable by an external tester without internal access to documentation or code (such as “black box” testing), although the lower number of requirements should make it easier to verify.

Level 2

Most applications should be striving to achieve this level of security. Around 50% of the requirements in the ASVS are L2 meaning that an application needs to implement around 70% of the requirements in the ASVS (all of the L1 and L2 requirements) in order to comply with L2.

These requirements generally relate to either less common attacks or more complicated protections against common attacks. They may still be a first layer of defense, or they may require certain preconditions for the attack to be successful.

Level 3

This level should be the goal for applications looking to demonstrate the highest levels of security and provides the final ~30% of requirements to comply with.

Requirements in this section are generally either defense-in-depth mechanisms or other useful but hard-to-implement controls.

Which level to achieve

The priority-based levels are intended to provide a reflection of the application security maturity of the organization and the application. Rather than the ASVS prescriptively stating what level an application should be at, an organization should analyze its risks and decide what level it believes it should be at, depending on the sensitivity of the application and of course, the expectations of the application's users.

For example, an early-stage startup that is only collecting limited sensitive data may decide to focus on Level 1 for its initial security goals, but a bank may have difficulty justifying anything less than Level 3 to its customers for its online banking application.

How to use the ASVS**The structure of the ASVS**

The ASVS is made up of a total of around 350 requirements which are divided into 17 chapters, each of which is further divided into sections.

The aim of the chapter and section division is to simplify choosing or filtering out chapters and sections based on what is relevant for the application. For example, for a machine-to-machine API, the requirements in chapter V3 related to web frontends will not be relevant. If there is no use of OAuth or WebRTC, then those chapters can be ignored as well.

Release strategy

ASVS releases follow the pattern "Major.Minor.Patch" and the numbers provide information on what has changed within the release. In a major release, the first number will change, in a minor release, the second number will change, and in a patch release, the third number will change.

- Major release - Full reorganization, almost everything may have changed, including requirement numbers. Reevaluation for compliance will be necessary (for example, 4.0.3 -> 5.0.0).
- Minor release - Requirements may be added or removed, but overall numbering will stay the same. Reevaluation for compliance will be necessary, but should be easier (for example, 5.0.0 -> 5.1.0).

- Patch release - Requirements may be removed (for example, if they are duplicates or outdated) or made less stringent, but an application that complied with the previous release will comply with the patch release as well (for example, 5.0.0 -> 5.0.1).

The above specifically relates to the requirements in the ASVS. Changes to surrounding text and other content such as the appendices will not be considered to be a breaking change.

Flexibility with the ASVS

Several of the points described above, such as documentation requirements and the levels mechanism, provide the ability to use the ASVS in a more flexible and organization-specific way.

Additionally, organizations are strongly encouraged to create an organization- or domain-specific fork that adjusts requirements based on the specific characteristics and risk levels of their applications. However, it is important to maintain traceability so that passing requirement 4.1.1 means the same across all versions.

Ideally, each organization should create its own tailored ASVS, omitting irrelevant sections (e.g., GraphQL, WebSockets, SOAP, if unused). An organization-specific ASVS version or supplement is also a good place to provide organization-specific implementation guidance, detailing libraries or resources to use when complying with requirements.

How to Reference ASVS Requirements

Each requirement has an identifier in the format `<chapter>.<section>.<requirement>`, where each element is a number. For example, 1.11.3.

- The `<chapter>` value corresponds to the chapter from which the requirement comes; for example, all 1.#.# requirements are from the 'Encoding and Sanitization' chapter.
- The `<section>` value corresponds to the section within that chapter where the requirement appears, for example: all 1.2.# requirements are in the 'Injection Prevention' section of the 'Encoding and Sanitization' chapter.
- The `<requirement>` value identifies the specific requirement within the chapter and section, for example, 1.2.5 which as of version 5.0.0 of this standard is:

Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.

Since the identifiers may change between versions of the standard, it is preferable for other documents, reports, or tools to use the following format: `v<version>-<chapter>.<section>.<requirement>`, where: 'version' is the ASVS version tag. For example: `v5.0.0-1.2.5` would be understood to mean specifically the 5th requirement in the 'Injection Prevention' section of the 'Encoding and Sanitization' chapter from version 5.0.0. (This could be summarized as `v<version>-<requirement_identifier>`.)

Note: The `v` preceding the version number in the format should always be lowercase.

If identifiers are used without including the `v<version>` element then they should be assumed to refer to the latest Application Security Verification Standard content. As the standard grows and changes this becomes problematic, which is why writers or developers should include the version element.

ASVS requirement lists are made available in CSV, JSON, and other formats which may be useful for reference or programmatic use.

Forking the ASVS

Organizations can benefit from adopting ASVS by choosing one of the three levels or by creating a domain-specific fork that adjusts requirements per application risk level. This type of fork is encouraged, provided that it maintains traceability so that passing requirement 4.1.1 means the same across all versions.

Ideally, each organization should create its own tailored ASVS, omitting irrelevant sections (e.g., GraphQL, Websockets, SOAP, if unused). Forking should start with ASVS Level 1 as a baseline, advancing to Levels 2 or 3 based on the application's risk.

Use cases for the ASVS

The ASVS can be used to assess the security of an application and this is explored in more depth in the next chapter. However, several other potential uses for the ASVS (or a forked version) have been identified.

As Detailed Security Architecture Guidance

One of the more common uses for the Application Security Verification Standard is as a resource for security architects. There are limited resources available for how to build a secure application architecture, especially with modern applications. ASVS can be used to fill in those gaps by allowing security architects to choose better controls for common problems, such as data protection patterns and input validation strategies. The architecture and documentation requirements will be particularly useful for this.

As a Specialized Secure Coding Reference

The ASVS can be used as a basis for preparing a secure coding reference during application development, helping developers to make sure that they keep security in mind when they build software. Whilst the ASVS can be the base, organizations should prepare their own specific guidance which is clear and unified and ideally be prepared based on guidance from security engineers or security

architects. As an extension to this, organizations are encouraged wherever possible to prepare approved security mechanisms and libraries that can be referenced in the guidance and used by developers.

As a Guide for Automated Unit and Integration Tests

The ASVS is designed to be highly testable. Some verifications will be technical where as other requirements (such as the architectural and documentation requirements) may require documentation or architecture review. By building unit and integration tests that test and fuzz for specific and relevant abuse cases related to the requirements that are verifiable by technical means, it should be easier to check that these controls are operating correctly on each build. For example, additional tests can be crafted for the test suite for a login controller, testing the username parameter for common default usernames, account enumeration, brute forcing, LDAP and SQL injection, and XSS. Similarly, a test on the password parameter should include common passwords, password length, null byte injection, removing the parameter, XSS, and more.

For Secure Development Training

ASVS can also be used to define the characteristics of secure software. Many “secure coding” courses are simply ethical hacking courses with a light smear of coding tips. This may not necessarily help developers to write more secure code. Instead, secure development courses can use the ASVS with a strong focus on the positive mechanisms found in the ASVS, rather than the Top 10 negative things not to do. The ASVS structure also provides a logical structure for walking through the different topics when securing an application.

As a Framework for Guiding the Procurement of Secure Software

The ASVS is a great framework to help with secure software procurement or procurement of custom development services. The buyer can simply set a requirement that the software they wish to procure must be developed at ASVS level X, and request that the seller proves that the software satisfies ASVS level X.

Applying ASVS in Practice

Different threats have different motivations. Some industries have unique information and technology assets and domain-specific regulatory compliance requirements.

Organizations are strongly encouraged to look deeply at their unique risk characteristics based on the nature of their business, and based upon that risk and business requirements determine the appropriate ASVS level.

Assessment and Certification

OWASP's Stance on ASVS Certifications and Trust Marks

OWASP, as a vendor-neutral nonprofit, does not certify any vendors, verifiers, or software. Any assurance, trust mark, or certification claiming ASVS compliance is not officially endorsed by OWASP, so organizations should be cautious of third-party claims of ASVS certification.

Organizations may offer assurance services, provided they do not claim official OWASP certification.

How to Verify ASVS Compliance

The ASVS is deliberately not prescriptive about exactly how to verify compliance at the level of a testing guide. However, it is important to highlight some key points.

Verification reporting

Traditional penetration testing reports issues “by exception,” only listing failures. However, an ASVS certification report should include scope, a summary of all requirements checked, the requirements where exceptions were noted, and guidance on resolving issues. Some requirements may be non-applicable (e.g., session management in stateless APIs), and this must be noted in the report.

Scope of Verification

An organization developing an application will generally not implement all requirements, as some may be irrelevant or less significant based on the functionality of the application. The verifier should make the scope of the verification clear including which Level the organization is attempting to achieve and which requirements were included. This should be from the perspective of what was included rather than what was not included. They should also provide an opinion on the rationale of excluding the requirements which haven't been implemented.

This should allow the consumer of a verification report to understand the context of the verification and make an informed decision about the level of trust they can place in the application.

Certifying organizations can choose their testing methods but should disclose them in the report and this should ideally be repeatable. Different methods, like manual penetration tests or source code analysis, may be used to verify aspects such as input validation, depending on the application and requirements.

Verification Mechanisms

There are a number of different techniques which may be needed to verify specific ASVS requirements. Aside from penetration testing (using valid credentials to get full application coverage), verifying ASVS requirements may require access to documentation, source code, configuration, and the people involved in the development process. Especially for verifying L2 and L3 requirements. It is standard practice to provide robust evidence of findings with detailed documentation, which may include work papers, screenshots, scripts, and testing logs. Merely running an automated tool without thorough testing is insufficient for certification, as each requirement must be verifiably tested.

The use of automation to verify ASVS requirements is a topic that is constantly of interest. It is therefore important to clarify some points related to automated and black box testing.

The Role of Automated Security Testing Tools When automated security testing tools such as Dynamic and Static Application Security Testing tools (DAST and SAST) are correctly implemented in the build pipeline, they may be able to identify some security issues that should never exist. However, without careful configuration and tuning they will not provide the required coverage and the level of noise will prevent real security issues from being identified and mitigated.

Whilst this may provide coverage of some of the more basic and straightforward technical requirements such as those relating to output encoding or sanitation, it is critical to note that these tools will be unable entirely to verify many of the more complicated ASVS requirements or those that relate to business logic and access control.

For less straightforward requirements, it is likely that automation can still be utilized but application specific verifications will need to be written to achieve this. These may be similar to unit and integration tests that the organization may already be using. It may therefore be possible to use this existing test automation infrastructure to write these ASVS specific tests. Whilst doing this will require short term investment, the long term benefits being able to continually verify these ASVS requirements will be significant.

In summary, testable using automation != running an off the shelf tool.

The Role of Penetration Testing Whilst L1 in version 4.0 was optimized for “black box”(no documentation and no source) testing to occur, even then the standard was clear that it is not an effective assurance activity and should be actively discouraged.

Testing without access to necessary additional information is an inefficient and ineffective mechanism for security verification, as it misses out on the possibility of reviewing the source, identifying threats and missing controls, and performing a far more thorough test in a shorter timeframe.

It is strongly encouraged to perform documentation or source code-led (hybrid) penetration testing, which have full access to the application developers and the application’s documentation, rather than traditional penetration tests. This will certainly be necessary in order to verify many of the ASVS requirements.